

PYTHON

Dr. **Bülent Çobanoğlu**

cobanoglubulent[at]gmail.com

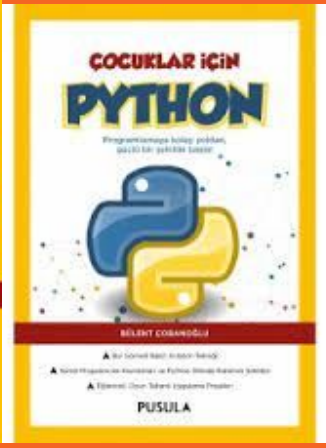
Ben kimim?

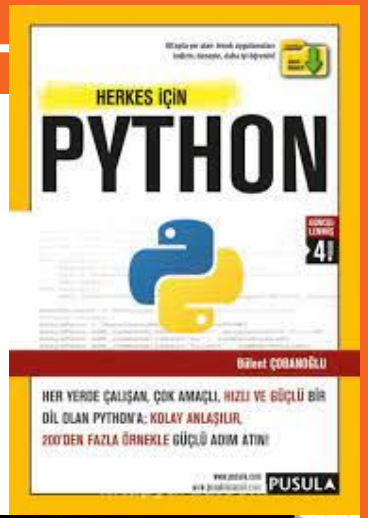
Kaynaklar;

- Eğitmen notları, https://github.com/bcobanoglu
- Python resmi sitesi

https://docs.python.org/3/









İçindekiler

- **random modulü**
- List- Array Farkı
- List metotları
- Tuple,
- Set
- Frozenset,
- Dict,
- Nested Dict
- Unpacking,
- dict üreteci
- zip() fonksiyonu

- match - case
- Döngüler
- string modülü
*** İndis Kavramı**
- String~Liste Dilimleme
- Listeler
- Liste üreteçleri

Dünkü derste yeni öğrendiğiniz iki şeyi yazabilir misiniz?

1

2

match case yapısı

Python 3.10 sürümü ile birlikte diğer dillerdeki 'switch-case' yapısına benzer ama çok daha yetenekli 'match-case' desen eşleştirme (pattern matching) yapısı dile eklenmiştir.

```
match desen:
    case <koşul_1>:  #1. eşleşme de işlem-1
            <işlem-1>
    case <koşul_2>:  #2. eşleşme de işlem-2
            <işlem-2>
    case <kosul_3>: #3. eşleşme de işlem-3
            <işlem-3>
    case _:                 # else durumu
            <işlem-4>
```

match case yapısı

Bir web sitesine girişte karşılaşılabilecek HTTP istemci hata kodlarını gösteren bir program;

```
status = int(input("Web page status number..:"))
match status:
  case 400:
      print("Bad request")
  case 404:
      print("Not found")
  case 401 | 403:  # 401 veya 403 ise
      print("Not allowed")
  case 418:
      print("I'm a teapot")
  case s if 200 <= status <= 299:
      print(f"{s}: Success")
  case _:  # else
      print("Something's wrong with the internet")
```

Döngüler (Loops)

- Hemen hemen bütün programlama dillerinde olduğu gibi Python dilinde de **iki farklı döngü yapısı** vardır. Bunlar;

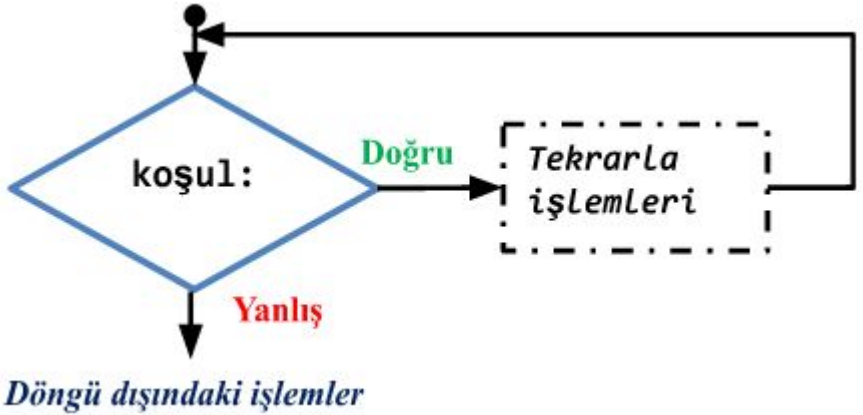
  - `while`
  - `for`

https://docs.python.org/3/tutorial/controlflow.html

while

- İşlemlerin ne kadar **tekrarlanacağının koşulun doğru** ya da **yanlış** olmasına bağlı olduğu, **koşul geçerli olduğu sürece** işlemlerin tekrarlandığı döngülerdir.

```
while (koşul):
    #işlemler
    ...
```

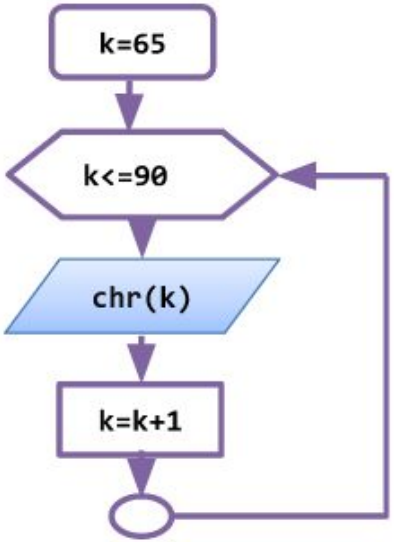


while

- A dan Z ye kadar İngilizce karakterleri ekranda gösteren programı while döngüsü ile kodlayalım. Programın örnek ekran çıktısı;

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Algoritması



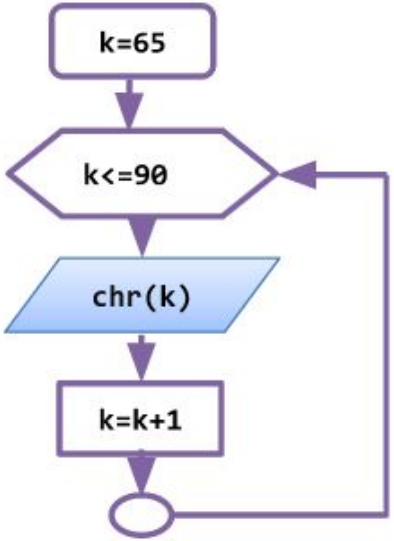
ASCII Tablosu

A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
K	75
L	76
M	77
N	78
O	79
P	80
Q	81
R	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

while

- A dan Z ye kadar İngilizce karakterleri ekranda gösteren programı while döngüsü ile kodlayalım.

Algoritması




```
1   k = 65
2   while k<=90:
3       #chr(65) = 'A' demektir
4       print (chr(k), end=' ')
5       k = k + 1
```

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
```

while True:

- Sonsuz döngü oluşturur. Döngü içerisindekiler koşulsuz çalıştırılır. Döngüden çıkış ancak **break** komutu ile döngü kırılarak gerçekleşir

```
Farklı Sonsuz Döngü Örnekleri:
  while True:
          ….
  while(1):
          ….
  while(10):
          ….
  while(-10):
          ….
```

while True Kullanımı

- Kullanıcı (user) şifreyi doğru girerse ATM ekranına ‘hoşgeldiniz’ mesajını veren **değilse** sürekli şifreyi doğru girmesini bekleyen bir sorgu ekranını while döngüsü ile kodlayalım…

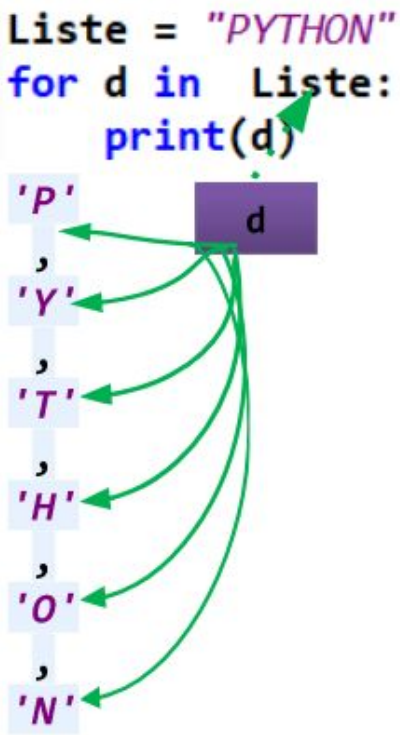
```
while True:
  sifre = input("Şifrenizi girin: ")
  if sifre == "1234":
      print("Hoşgeldiniz") break

print("Hatalı şifre! Tekrar deneyin.")
```

for

- Tekrar sayısının **baştan belli olduğu,** ardışık eleman listesinin **sırayla işleme konulduğu** döngü yapısıdır. for döngüsü **in** operatörünü kullanarak liste üzerinde ardışık (iteratif) ilerler.

```
for d in  Liste:
    #işlem1
    #işlem2
    #....
```



for

- **range()** komutu ile ardışık bir şekilde sayılar listelendiğine göre **for döngü** yapısında doğrudan **liste** kullanılabilir mi?

```
1   for a in range(0,5):
2       print("Python")
```

Eşdeğeri

```
1   for a in [0,1,2,3,4]:
2       print("Python")
```

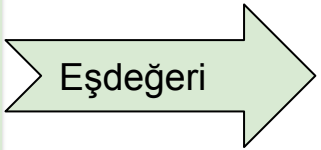
```
Python
Python
Python
Python
Python
```

```
Python
Python
Python
Python
Python
```

for - while eşdeğeri:

- 0' dan 20' e kadar ki (*20 dahil*) **çift sayıların toplamını** ekranda gösteren programı hem **for** hemde **while** döngüsü ile kodlayalım.

```
T = 0
for a in range(0,22,2):
    T = T + a
#döngü sonu
print("Toplam = ", T)
```




```
a = 0
T = 0 #Toplama değişkeni
while a < 22:
   T = T + a #Yığmalı toplama
   a = a + 2
#Döngü sonu
print ("Toplam =", T)
```

break deyimi

- Bir şarta bağlı olarak (**sonsuz döngü dahil**) herhangi bir döngüyü **sonlandırmak** için kullanılır.

```
1 for a in range(0,12,2):
2     if (a==6):
3         break
4     print(a, end=' ')
```

Console

<terminated> printKomut.py [C:\Users\bc\

```
0 2 4
```

continue deyimi

- Bir şarta bağlı olarak **döngü başı yapmak** için kullanılır.

- **break** program akışını döngü sonrası komuta,
- **continue** ise döngü başına getirir.

```
1 for a in range(0,12,2):
2     if (a==6):
3         continue
4     print(a, end=' ')
```

Console

<terminated> printKomut.py [C:\Users\bc\

0 2 4 8 10

continue deyimi

- Bir şarta bağlı olarak **döngü başı yaparak, kaldığı yerden** devam eder. Aşağıdaki programın çıktısında hangisi görülmez?

```
asansor_nums = list(range(1,5))
for i in asansor_nums:
  if i==3:
      continue # alttaki kodu çalıştırmadan döngüye devam eder.
  print(f"{i}. kat")
```



Students choose an option

Pear Deck Interactive Slide
Do not remove this bar

Döngülerde else kullanımı

- Aynı **if-else** de olduğu gibi while/for döngülerinde de **else** kullanılabilir mi?

```
print("6 ya kaç eklersem 10 eder?")
a = 6
t = 0
while a < 10:
    a += t
    print(f"Hayır {t} eklenirse 10 olmaz")
    t += 1
else:
    print(f"Evet {t} eklenirse 10 olur")
```

İç içe Döngüler (Nested Loops)

- Bir döngü yapısı içerisinde başka bir döngü kullanılarak ***iç içe döngüler*** oluşturulabilir. İç içe döngülere;
  - **Zaman sistemi**;
  - **Tablo şeklindeki (satır ve sütunlardan oluşan) çarpım tablosu gibi yapılar/desenler;**
  - **Çok boyutlu diziler;**

örnek gösterilebilir.

İç içe Döngüler (Nested Loops)

- **Task** 👇 Aşağıdaki çarpım tablosunu nasıl kodlarız?

```
1 * 1 = 1       2 * 1 = 2       3 * 1 = 3       4 * 1 = 4       5 * 1 = 5       6 * 1 = 6       7 * 1 = 7       8 * 1 = 8       9 * 1 = 9       10 * 1 = 10
1 * 2 = 2       2 * 2 = 4       3 * 2 = 6       4 * 2 = 8       5 * 2 = 10      6 * 2 = 12      7 * 2 = 14      8 * 2 = 16      9 * 2 = 18      10 * 2 = 20
1 * 3 = 3       2 * 3 = 6       3 * 3 = 9       4 * 3 = 12      5 * 3 = 15      6 * 3 = 18      7 * 3 = 21      8 * 3 = 24      9 * 3 = 27      10 * 3 = 30
1 * 4 = 4       2 * 4 = 8       3 * 4 = 12      4 * 4 = 16      5 * 4 = 20      6 * 4 = 24      7 * 4 = 28      8 * 4 = 32      9 * 4 = 36      10 * 4 = 40
1 * 5 = 5       2 * 5 = 10      3 * 5 = 15      4 * 5 = 20      5 * 5 = 25      6 * 5 = 30      7 * 5 = 35      8 * 5 = 40      9 * 5 = 45      10 * 5 = 50
1 * 6 = 6       2 * 6 = 12      3 * 6 = 18      4 * 6 = 24      5 * 6 = 30      6 * 6 = 36      7 * 6 = 42      8 * 6 = 48      9 * 6 = 54      10 * 6 = 60
1 * 7 = 7       2 * 7 = 14      3 * 7 = 21      4 * 7 = 28      5 * 7 = 35      6 * 7 = 42      7 * 7 = 49      8 * 7 = 56      9 * 7 = 63      10 * 7 = 70
1 * 8 = 8       2 * 8 = 16      3 * 8 = 24      4 * 8 = 32      5 * 8 = 40      6 * 8 = 48      7 * 8 = 56      8 * 8 = 64      9 * 8 = 72      10 * 8 = 80
1 * 9 = 9       2 * 9 = 18      3 * 9 = 27      4 * 9 = 36      5 * 9 = 45      6 * 9 = 54      7 * 9 = 63      8 * 9 = 72      9 * 9 = 81      10 * 9 = 90
1 * 10 = 10     2 * 10 = 20     3 * 10 = 30     4 * 10 = 40     5 * 10 = 50     6 * 10 = 60     7 * 10 = 70     8 * 10 = 80     9 * 10 = 90     10 * 10 = 100
```

İç içe Döngüler (Nested Loops)

- **Task:** Aşağıdaki çarpım tablosunu nasıl kodlarız?

```
for A in range(1,11):
    for B in range(1,11):
        print (B,'*', A,'=',A * B, end="\t")
    print() #bir satir atla
    #B dongusu sonu
#A dongusu sonu
```

```
1 * 1 = 1	2 * 1 = 2	3 * 1 = 3	4 * 1 = 4	5 * 1 = 5	6 * 1 = 6	7 * 1 = 7	8 * 1 = 8	9 * 1 = 9	10 * 1 = 10
1 * 2 = 2	2 * 2 = 4	3 * 2 = 6	4 * 2 = 8	5 * 2 = 10	6 * 2 = 12	7 * 2 = 14	8 * 2 = 16	9 * 2 = 18	10 * 2 = 20
1 * 3 = 3	2 * 3 = 6	3 * 3 = 9	4 * 3 = 12	5 * 3 = 15	6 * 3 = 18	7 * 3 = 21	8 * 3 = 24	9 * 3 = 27	10 * 3 = 30
1 * 4 = 4	2 * 4 = 8	3 * 4 = 12	4 * 4 = 16	5 * 4 = 20	6 * 4 = 24	7 * 4 = 28	8 * 4 = 32	9 * 4 = 36	10 * 4 = 40
1 * 5 = 5	2 * 5 = 10	3 * 5 = 15	4 * 5 = 20	5 * 5 = 25	6 * 5 = 30	7 * 5 = 35	8 * 5 = 40	9 * 5 = 45	10 * 5 = 50
1 * 6 = 6	2 * 6 = 12	3 * 6 = 18	4 * 6 = 24	5 * 6 = 30	6 * 6 = 36	7 * 6 = 42	8 * 6 = 48	9 * 6 = 54	10 * 6 = 60
1 * 7 = 7	2 * 7 = 14	3 * 7 = 21	4 * 7 = 28	5 * 7 = 35	6 * 7 = 42	7 * 7 = 49	8 * 7 = 56	9 * 7 = 63	10 * 7 = 70
1 * 8 = 8	2 * 8 = 16	3 * 8 = 24	4 * 8 = 32	5 * 8 = 40	6 * 8 = 48	7 * 8 = 56	8 * 8 = 64	9 * 8 = 72	10 * 8 = 80
1 * 9 = 9	2 * 9 = 18	3 * 9 = 27	4 * 9 = 36	5 * 9 = 45	6 * 9 = 54	7 * 9 = 63	8 * 9 = 72	9 * 9 = 81	10 * 9 = 90
1 * 10 = 10	2 * 10 = 20	3 * 10 = 30	4 * 10 = 40	5 * 10 = 50	6 * 10 = 60	7 * 10 = 70	8 * 10 = 80	9 * 10 = 90	10 * 10 = 100
```

İç içe döngüler

```
calisan = ["Ali ", "Hilal ", "Pınar "]
mod = ["mutlu", "mutsuz"]
for i in calisan:   # dış döngü
    for ii in mod:  # iç döngü
        print(i + ii)
```

✓ 0.0s

```
Ali mutlu
Ali mutsuz
Hilal mutlu
Hilal mutsuz
Pınar mutlu
Pınar mutsuz
```

Q:
Aşağıdaki metindeki emojileri ve özel karakterleri nasıl temizleyebiliriz?



```
mesaj = "+Python Dersi! kolay mı yoksa zor mu? [😦|😅]"
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Q:
Aşağıdaki metindeki emojileri ve özel karakterleri nasıl temizleyebiliriz?



```
1 mesaj = "+Python Dersi! kolay mi yoksa zor mu? [😢|😅] "
2
3 for i in mesaj:
4     if i.isalnum() or i.isspace():
5         print(i,end="")
```

```
Python Dersi kolay mi yoksa zor mu
```

string Modülü

`string.`**`ascii_letters`**
The concatenation of the `ascii_lowercase` and `ascii_uppercase` constants described below. This value is not locale-dependent.

`string.`**`ascii_lowercase`**
The lowercase letters `'abcdefghijklmnopqrstuvwxyz'`. This value is not locale-dependent and will not change.

`string.`**`ascii_uppercase`**
The uppercase letters `'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. This value is not locale-dependent and will not change.

`string.`**`digits`**
The string `'0123456789'`.

`string.`**`hexdigits`**
The string `'0123456789abcdefABCDEF'`.

`string.`**`octdigits`**
The string `'01234567'`.

`string.`**`punctuation`**
String of ASCII characters which are considered punctuation characters in the `C` locale: `!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~`.

`string.`**`printable`**
String of ASCII characters which are considered printable. This is a combination of `digits`, `ascii_letters`, `punctuation`, and `whitespace`.

`string.`**`whitespace`**
A string containing all ASCII characters that are considered whitespace. This includes the characters space, tab, linefeed, return, formfeed, and vertical tab.

https://docs.python.org/3/library/string.html

random Modulü

- Rastgele sayı üretimi için **random** modülüne ait **random(), randrange(), randint(), uniform()** gibi fonksiyonlar kullanılabilir.

Fonksiyon	Açıklama
`random.random()`	**'0 - 1'** arasında **float** tipinde rastgele bir sayı üretir.
`random.randint(s1,s2)`	**'s1 - s2'** arasında **int** tipinde rastgele bir sayı üretir. (*s2 dahil).*
`random.randrange(s1,s2,adım)`	**'s1-s2'** değerleri arasında adım miktarına dikkat edilerek **float** tipinde rasgele bir sayı üretir, adım belirtilmezse 1 kabul edilir. (*s2 dahil değil)*
`random.uniform(s1,s2)`	**'s1 - s2'** arasında **float** tipinde rasgele bir sayı üretir
`random.choice([Liste])`	Listeden rastgele bir eleman seçer.
`random.shuffle([Liste])`	Listeyi rastgele sıralar.
`random.sample([Liste], a)`	Listeden rastgele **a adet eleman** seçer.

https://docs.python.org/3.10/library/random.html

Task-Home: En az bir adet küçük karakter, bir adet büyük karakter, bir adet özel karakter ve bir adet rakam içeren bir parola oluşturucu programı kodlayınız.



```
Zq[iX69_
```

Password Üreteçi

- En az bir adet küçük karakter, bir adet büyük karakter, bir adet özel karakter ve bir adet rakam içeren bir parola oluşturucu programı kodlayınız.

```
import string
import random
kucuk_harf = random.sample(string.ascii_lowercase,2)
buyuk_harf = random.sample(string.ascii_uppercase,2)
digits = random.sample(string.digits,2)
ozel_karakter = random.sample(string.punctuation,2)
sifre = kucuk_harf+buyuk_harf+digits+ozel_karakter
random.shuffle(sifre)
print("".join(sifre))
```

✓ 0.0s

```
3b[2tJ\Q
```

Loto: Listeden rastgele sayı seçme

- Bilgisayarın rastgele ürettiği 1-49 arasından 6 adet sayıyı rastgele seçen (sayısal loto çekiliş sonucu olarak ekrana yazan) programı kodlayalım.

```
1  import random
2  listem = range(random.randrange(1,50))
3  new_list = random.sample(listem, k=6)
4  new_list
```

✓ 0.0s

```
[5, 12, 20, 2, 25, 33]
```

Sayısal Loto: Döngülü Çözüm

- Bilgisayarın rastgele ürettiği 1-49 arasındaki 6 adet sayıyı sayısal loto çekiliş sonucu olarak ekrana yazan programı kodlayalım.

```
import random
random.seed(103)
print("Sayısal Loto Çekilişi\n")
#6 adet random sayı üret
for i in range (6):
    sL=random.randint(1,49)
    #sayıları yan yana yaz
    print (sL, end=' ')
```

Sayı Tahmini Oyunu

- Bilgisayarın tuttuğu (0-100 arasındaki) sayıya göre kullanıcıyı “daha küçük veya daha büyük sayı giriniz gibi” yönlendiren ve kaç tahminde sayıyı bulduğunu ve puanını söyleyen bir sayı tahmin oyununu kodlayalım.

```
import random
sayi = random.randint(1,100)
sayac=0 ; puan=100
print("İlk Tahmininiz.:");
while True:
    tahmin =int(input())
    sayac = sayac + 1
    if tahmin == sayi:
        print("Bravo!",sayac,".de bildiniz")
        break #döngüden çık
    elif tahmin < sayi:
        print("Daha büyük bir sayı gir!")
    else:
        print("Daha küçük bir sayı gir!")
    puan = puan - 10
print("Puanınız.:",puan)
```

```
İlk Tahmininiz.:
44
Daha büyük bir sayı gir!
66
Daha büyük bir sayı gir!
88
Daha küçük bir sayı gir!
77
Daha küçük bir sayı gir!
71
Daha küçük bir sayı gir!
69
Daha küçük bir sayı gir!
68
Bravo! 7 .de bildiniz
Puanınız.: 40
```

İndis Kavramı

- String işlemlerde veya listelerde/dizilerde değişken ismini takiben köşeli parantez **[***indis_no***]** içerisinde belirtilen numaraya **indis** adı verilir. Bir stringin (karakter dizisinin) veya listenin her bir elemanına indis numarası ile erişilir.

ad = "	M	e	r	h	a	b	a		A	l	i	"
pozitif indis	0	1	2	3	4	5	6	7	8	9	10	
negatif indis	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	

İndis Kavramı

Ad = "	M	e	r	h	a	b	a		A	l	i	"
pozitif indis	0	1	2	3	4	5	6	7	8	9	10	
negatif indis	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	

```
>>> Ad = "Merhaba Ali"
>>> Ad[0]
'M'
>>> Ad[-11]
'M'
>>> Ad[2]
'r'
>>> Ad[-1]
'i'
```

index() ile indisi öğrenme

- Bir karakterin indis numarasını öğrenmek için **`index()`** metodu kullanılabilir;

```
ad = "Merhaba Ali"
ad.index('h')
```

✓ 0.0s

```
3
```

`index(item)` Belirtilen öğenin konumunu döndürür. `lis = [2,3]; lis.index(2) # out: 0`

Dilimleme(Slicing)

- Slicing, bir liste, dizi veya stringlerde belirli bir bölümü almak, dilimlemek için kullanılan bir yöntemdir. Bu amaçla **":"** operatörü kullanılır.
- Kullanım şekli;

```
değişken [ilkIndis: sonIndis: adımMiktari]
```

Dilimleme(Slicing)

: Kullanımı	Açıklama	Ekran Çıktısı
`S[:]`	Karakter dizisini olduğu gibi al.	`Python?`
`S[3:]`	3. indisli (4.) elemandan dizi sonuna kadar al.	`hon?`
`S[:3]`	Başlangıçtan itibaren ilk 3 elemanı al.	`Pyt`
`S[:-3]`	Başlangıçtan itibaren -3. indisli(4. indisli) elemana kadar al.	`Pyth`
`S[1::2]`	1.indisli (2.sıradaki) elemandan başla ve ikişer atlayarak dizi sonuna git.	`yhn`
`S[1:6:3]`	1.indisli (2.sıradaki) elemandan başla ve 6.indisli elemana kadar 3'er 3'er git.	`yo`
`S[::-1]`	Karakter dizisini TERS ÇEVİR	`?nohtyP`

S = "	P	y	t	h	o	n	?	"
pozitif indis	0	1	2	3	4	5	6	
negatif indis	-7	-6	-5	-4	-3	-2	**-1**	

Dilimleme(Slicing)

- Slicing kullanım şekli ve örnekleri;

değişken **[ilkIndis: sonIndis: adım/atlama sayısı]**

```
my_list = [10, 20, 30, 40, 50]
my_str = "Merhaba!"

# Başlangıç ve bitiş inddisleri ile slicing
print(my_list[1:4]) # [20, 30, 40]
print(my_str[2:6])  # "rhab"

# Atlama parametresi kullanarak slicing
print(my_list[::2]) # [10, 30, 50]
print(my_str[::3])  # "Mha"
print(my_str[::-2])  # "!bhe"
```

Q: 👇 String dilimleme

⏩ Normalde string elemanları doğrudan değiştirilemez **(immutable)** özelliktedir. Buna göre '**python**' stringinin 3.indisli karakterini nasıl kaldırabilirsiniz/silebilirsiniz?

```
pyton
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Task: 👇 String dilimleme

⏩ Normalde string elemanları doğrudan değiştirilemez **(immutable)** özelliktedir. Buna göre '**python**' stringinin 3.indisli karakterini nasıl kaldırabilirsiniz/silebilirsiniz?

```
word = 'python'
n = 3
head = word[:n]
tail = word[n+1:]
print(head + tail)
```

Eşdeğeri

```
word = 'python'
n = 3
w_list = list(word)
w_list[n]=""
"".join(w_list)
```

Eşdeğeri

```
word = 'python'
t = word.replace('h','')
```

```
pyton
```

Stringi Ters Çevirme: Palindrom

⏩ Girilen bir metnin **Palindrom** olup olmadığını sorgulayan programı kodlayalım.

```
s = input("Bir metin gir.:")
if s == s[::-1]:
#''.join(reversed(s)):
    print(s, ":palindrom")
else:
    print(s, ":palindrom degil")
```

Liste (list) Oluşturma

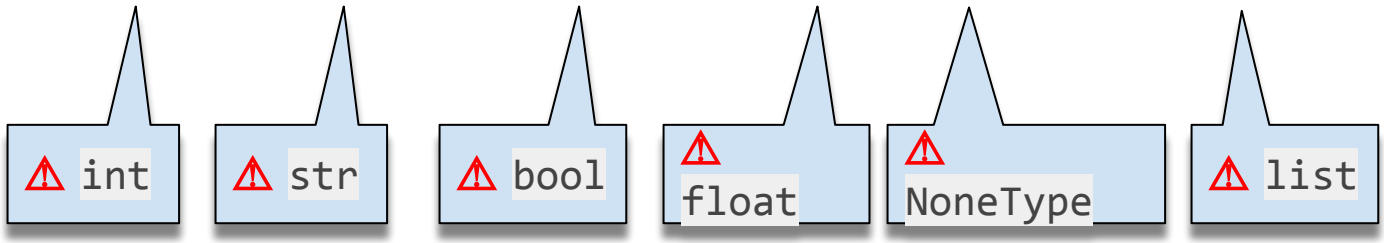
- **Sıralı** eleman dizisidir. Liste “Python” gibi bir karakter dizisi olabileceği gibi “0,1,2,3,4,5” gibi bir sayı dizisi de olabilir. Sayı dizisi elde etmek için **range()** fonksiyonu veya **numpy** modülünün metotları kullanılabilir.
- Liste (list) oluşturmanın iki yolu var;

```
[]
list()
```

Liste (list) Oluşturma

- Listeler farklı veri tipindeki elemanlardan oluşabilir.
- Örneğin :

```
mix_list = [13, 'Yusuf', True, 3.145, None, [1, 2, 3]]
```



Liste (list) Oluşturma

- **Q:👇**
  - `"Merhaba 2023"` stringini listeye dönüştürünüz
  - Hem `list()` fonksiyonunu hemde `[]` parantezleri kullanınız.



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Liste (list) Oluşturma

- Task:
  - `"Merhaba 2023"` stringini listeye dönüştürünüz
  - Hem `list()` fonksiyonunu hemde `[]` parantezleri kullanınız.

```
1   text = "Merhaba 2023"
2   list0 = [text]
3   list1 = list(text)
4   print(list0)
5   print(list1)
```

```
['Merhaba 2023']
['M', 'e', 'r', 'h', 'a', 'b', 'a', ' ', '2', '0', '2', '3']
```

range ile Liste Oluşturma

- range:
  - İstenilen aralıkta sayı dizisi oluşturmak için kullanılır. Aynı zamanda for döngü yapısında **tekrar sayısını belirtmek** için kullanılan bir fonksiyondur. Kullanım şekli:
  - **range(**ilk, son, adım**)**

```
>>> range(5)
[0, 1, 2, 3, 4] #şeklinde 0-5 (5 hariç) arasındaki sayıları üretir.
>>> range(1, 6)
[1, 2, 3, 4, 5] # (1 dahil, 6 hariçtir)
>>> range(3, 30, 3)
[3, 6, 9, 12, 15, 18, 21, 24, 27]
>>> range(4, 1, -1)
[4, 3, 2] # sayılarını geriye doğru üretir
```

Liste ile Array(Dizi) arasında bir fark var mıdır?



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

List-Array(Dizi) Farkı

- Array'ler aynı tip elemanlardan oluşurken, listeler farklı tip elemanlardan oluşabilir.

```
Liste = ['A', 1, [1,2,3], "Nar", 3.14]
```

```
import numpy as np
Dizi = np.array([2,3,4,5,6,7,8,9])
```

Liste kopyalama

`copy()` Listenin bir kopyasını döndürür. `lis = [2,4,3]; c = lis.copy() # c: [2,4,3]`

Bu iki program arasındaki fark nedir?

```
lis = [2,4,3]
copy_lis = lis
copy_lis.append(7)
print(lis)
```

✓ 0.0s

```
[2, 4, 3, 7]
```

```
lis = [2,4,3]
copy_lis = lis.copy()
copy_lis.append(7)
print(lis)
```

✓ 0.0s

```
[2, 4, 3]
```

Listeye Eleman Ekleme

- Bir listeye eleman eklemek için;

```
.append()
.extend()  # +
.insert()
```

`append(item)`	Listenin sonuna bir öğe ekler.	`lis = [2,3]; lis.append(4) # out: [2,3,4]`
`extend(iterable)`	Listenin sonuna liste ekler.	`lis = [2,3]; lis.extend([4,5]) # out: [2,3,4,5]`
`insert(index, item)`	Belirtilen konuma öğe ekler.	`lis = [2,3]; lis.insert(1,4); # out: [2,4,3]`

for ile boş listeye eleman ekleme

- Bir listenin sonuna "**append()**" fonksiyonu ile yeni elemanlar eklenebilir. Kullanım şekli;

  liste.append(9) *# listenin sonuna 9 sayısını ekler.*

- Liste elemanları arasına yeni bir eleman eklemek için "**insert()**" fonksiyonu kullanılır. Kullanım şekli; liste.**insert**(2, **9**) *#listenin 2. indisini 9 yapar ve listeyi kaydırır.*

```
L = []
for i in range(10):
    L.append(i)

print(L)
```

✓ 0.0s

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Listeye Eleman Ekleme



- Çıktısı nedir?
- 👉

```
lis = []
lis.append(3)
lis.append("a")
lis.append([1,2])
print(lis)
```

Listeye Eleman Ekleme



- Çıktısı nedir?

```
lis = []
lis.append(3)
lis.append("a")
lis.append([1,2])
print(lis)
```

```
[3, 'a', [1, 2]]
```

Q: 👇 Listeye eleman ekleme

⏩ Çıktısı nedir?

```
x = ['a', 'b', 'c']
for i in range(len(x)):
    x.append(i)
print(x)
```

A. ['a', 'b', 'c', 0, 1, 2]

B. ['a', 'b', 'c', '0', '1', '2']

C. ['a', 'b', 'c']

D. Error

Students choose an option

Pear Deck Interactive Slide
Do not remove this bar

Listeye Eleman Ekleme



- Çıktısı nedir?

```
lis = []
lis.append(3)
lis.extend("a")
lis.extend([1,2])
print(lis)
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Listeye Eleman Ekleme



- Çıktısı nedir?

```
lis = []
lis.append(3)   # lis.extend(3) ?
lis.extend("a")
lis.extend([1,2])
print(lis)
```

```
[3, 'a', 1, 2]
```

Listeye Eleman Ekleme



- Çıktısı nedir?

```
lis = [1,3,5]
lis.insert(2,4)
print(lis)
lis.insert(2,[1,2])
print(lis)
```

Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Listeye Eleman Ekleme



- Çıktısı nedir?

```
lis = [1,3,5]
lis.insert(2,4)
print(lis)
lis.insert(2,[1,2])
print(lis)
```

```
[1, 3, 4, 5]
[1, 3, [1, 2], 4, 5]
```

Listeye Eleman Ekleme



- **`insert()`** ile liste sonuna bir eleman nasıl eklenir?

```
numbers = [3,4,5,6] #sonuna 7 nasıl eklenir?
```

```
# ipucu:
a.insert(len(a), x) is equivalent to a.append(x).
```



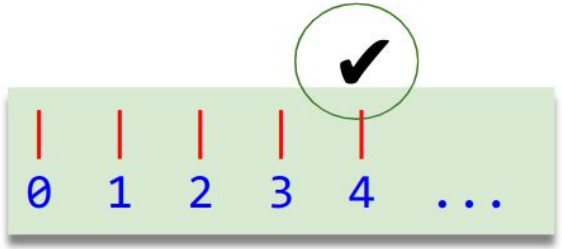
Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Listeye Eleman Ekleme



- **`insert()`** ile liste sonuna bir eleman nasıl eklenir?




```
numbers = [3, 4, 5, 6]
numbers.insert(4, 7)

print(numbers)
```

index

```
[3, 4, 5, 6, 7]
```

Listeye Eleman Ekleme



- **`insert()`** kullanmadan liste sonuna bir eleman nasıl eklenir?

```
numbers = [3, 4, 5, 6]
numbers[len(numbers):] = [7]
numbers
```

✓ 0.0s

```
[3, 4, 5, 6, 7]
```

Listeden Eleman Silme

- Bir listeden eleman çıkarmak için;

```
.remove()
.pop()
.clear()
```

`remove(item)`	Item'i listeden kaldırır.	`lis = [2,3]; lis.remove(3) # out: [2]`
`pop([index])`	indisi belirtile itemi siler ve geri döndürür.	`lis = [2,3]; sil = lis.pop(1) # sil:3`
`clear()`	Tüm öğeleri listeden kaldırır.	`lis = [2,3]; lis.clear() # out: []`

Task: Listeden eleman silme



- Bir listeden;
  - Belli bir elemanı silmek için **remove(**eleman**)**,
  - İndis numarası belirtilen (örneğin 3. indisli) bir elemanı silmek için L.**pop(3)**,
  - İndis numarası belirtilen belirli aralıktaki elemanları silmek için **del L[1:3]**,
  - Bir L listesini tamamen boşaltmak için L.**clear()** veya **del L[:]**

- **Buna göre** kullanıcının girdiği bir cümleyi listeye dönüştüren ve bu listeden sesli harfleri çıkaran (*sessiz harflerden bir liste oluşturan*) programı döngü kullanarak kodlayalım.

Task: Listeden eleman silme



- **Buna göre** kullanıcının girdiği bir cümleyi listeye dönüştüren ve bu listeden sesli harfleri çıkaran (*sessiz harflerden bir liste oluşturan*) programı döngü kullanarak kodlayalım.

```
sentence = "Ali ip at."
lis = list(sentence)
print(lis)

while "i" in lis: # bütün 'i' leri sil...
    lis.remove("i")

print(lis)

lis.remove("a")
print(lis)
```

✓ 0.0s

```
['A', 'l', 'i', ' ', 'i', 'p', ' ', 'a', 't', '.']
['A', 'l', ' ', 'p', ' ', 'a', 't', '.']
['A', 'l', ' ', 'p', ' ', 't', '.']
```

Task: Listeden eleman silme



- **Buna göre** kullanıcının girdiği bir cümleyi listeye dönüştüren ve bu listeden sesli harfleri çıkaran (*sessiz harflerden bir liste oluşturan*) programı döngü kullanarak kodlayalım.

```
sentence = "Ali ip at."
lis = list(sentence)
sessiz_lis = [i for i in lis if not i in 'aeiouıüö']
sessiz_lis
```

✓ 0.1s

```
['A', 'l', ' ', 'p', ' ', 't', '.']
```

Liste elemanlarını kopyalama

copy() Listenin öğelerini kopyalar. Eşdeğeri: **`liste[:]`**

```
nums = [12, 1, 2, 3, 0, 4]
copy_nums = nums.copy() #nums[:]
copy_nums.append(99)
print(copy_nums)
print(nums)
```

Bu iki program arasında bir fark varmıdır?

```
nums = [12, 1, 2, 3, 0, 4]
copy_nums = nums
copy_nums.append(99)
print(copy_nums)
print(nums)
```

Liste elemanlarını kopyalama

copy() Listenin öğelerini kopyalar. Eşdeğeri: **`liste[:]`**

```
nums = [12, 1, 2, 3, 0, 4]
copy_nums = nums.copy() #nums[:]
copy_nums.append(99)
print(copy_nums)
print(nums)
```

```
[12, 1, 2, 3, 0, 4, 99]
[12, 1, 2, 3, 0, 4]
```

```
nums = [12, 1, 2, 3, 0, 4]
copy_nums = nums
copy_nums.append(99)
print(copy_nums)
print(nums)
```

```
[12, 1, 2, 3, 0, 4, 99]
[12, 1, 2, 3, 0, 4, 99]
```

Liste elemanlarını sıralama

sort(key=None, reverse=False) Listenin öğelerini sıralar.

```
lis = [2,4,3]
lis.sort()

# out: [2, 3, 4]
```

```
lis = [2,4,3]
lis.sort(reverse=True)


# out: [4, 3, 2]
```

Liste elemanlarını sıralama

```
1  lis = [8, 4,"5","6", 9]
2  lis.sort()
```

⊗ 0.0s

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/home/bull/Downloads/ITF_PYTHON_WS_Strings_STUDENT.ipynb Cell 61 in ()
```

✓ Sıralanacak liste aynı tipten elemanlardan oluşmalı!!!

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

Liste elemanlarını sıralama

Q: Bir listenin sıralı hali başka bir değişkene nasıl aktarılır?

```
lis = [2,4,3]
```



```
[2, 3, 4]
```

Liste elemanlarını sıralama

✓ Bir listenin sıralı hali başka bir değişkene nasıl aktarılır?

```
lis = [2,4,3]
ters_lis = sorted(lis)
print(ters_lis)
```

✓ 0.0s

```
[2, 3, 4]
```

Liste elemanlarını ters çevirme

✓ `reverse()` : Listenin öğelerini tersine çevirir.

```
lis = [2,4,3]
lis.reverse()

# out: [3, 4, 2]
```




```
lis = [2,4,3]
lis[::-1]

# out: [4, 3, 2]
```

Liste elemanlarını ters çevirme

Q: Bir **listenin tersi** başka bir değişkene nasıl aktarılır?

```
lis = [2,4,3]
ters_lis = lis.reverse()
print(ters_lis)
```

✓ 0.0s

```
None
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Liste elemanlarını ters çevirme

✓ Bir listenin tersi veya sıralı hali bir değişkene nasıl aktarılır?
Why?

```
lis = [2,4,3]
ters_lis = reversed(lis)
print(ters_lis)
```

✓ 0.0s

```
<list_reverseiterator object at 0x7f11444bf010>
```

Listedeki eleman sayısı

`count()` Belirtilen öğenin listedeki tekrar sayısını döndürür.

```
lis = [2,3]; lis.count(2) # out: 1
```

`len()` Listenin eleman sayısını verir.

```
lis = list("Erik çiçeği")
print(len(lis))              #11
print(lis.count("ç"))        # 2
```

Liste dilimleme

- **lis1 ile lis2 arasında fark varmıdır?**

```
lis1 = list(range(1,101))
```

```
lis2 = [range(1,101)]
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Liste dilimleme

- 1-100 arası üretilen sayılardan tek olanları dilimleyip ekranda gösteren programı kodlayalım.

```
lis = list(range(1,101))
lis[0:101:2]     # lis[::2]
```

- **Q:** Aşağıda verilen karışık listede verilen stringleri dilimleme ile alabilir misiniz?



```
mix = [1, [1, "one", 2, "two", 3, "three", 4, "four"], 5, 6]
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

- **Task:** Aşağıda verilen karışık listede verilen stringleri dilimleme ile alabilir misiniz?



```
mix = [1, [1, "one", 2, "two", 3, "three", 4, "four"], 5, 6]

print(mix[1][1::2])
```

Tuple (Demet) Veri Yapısı

- **Virgülle ayrılmış** değerler parantez içine alınarak 👉 **`(tuple)`** oluşturulabilir.
- Tuple oluşturmanın bir başka yolu da **`tuple()`** sarmalına almaktır.

- `()`
- **`tuple()`**

```
1  word = "happy"
2  demet1 = tuple(word)
3  demet2 = ('h', 'a', 'p', 'p', 'y')
4  print(demet1)
5  print(demet2)
6  print(type(demet1))
```

✓ 0.0s

```
('h', 'a', 'p', 'p', 'y')
('h', 'a', 'p', 'p', 'y')
<class 'tuple'>
```

Q: Çıktıları nedir?

```
demet = ('a')
print(type(demet))
```

```
demet = ('a',)
print(type(demet))
```

```
demet = 'a',
print(type(demet))
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

List ile Tuple arasındaki fark?

`list`

- mutable
- [ ]
- Veri tipi: list
- Daha yavaş,
- Bellek tüketimi fazla,
- Çok fazla metoda sahip, 👉 dir(list)
- Eleman ekleme, silme işlemleri kolay
- Her ikisi de ordered(sıralı) veri tipi

`tuple`

- immutable
- (,)
- Veri tipi: tuple
- Daha hızlı,
- Bellek tüketimi az
- Az sayıda metoda sahip, 👉 dir(tuple)
- Eleman ekleme/silme işlemleri zor
- Her ikisi de ordered(sıralı) veri tipi
- dict'lerde key olarak kullanılabilir.

List ile Tuple arasındaki fark?

```
1  import timeit
2  start = timeit.default_timer() #kronometre başlat
3  t = (tuple(range(1,100000)), 'ali','emre','numan','musti','seba','dlim')
4  stop = timeit.default_timer()  #kronometre durdur
5  print((stop-start)*1000)
```

✓ 0.0s

```
2.456667000046764
```

```
1  import timeit
2  start = timeit.default_timer() #kronometre başlat
3  lis = [list(range(1,100000)), 'ali','emre','numan','musa','seba', 'clim']
4  stop = timeit.default_timer()  #kronometre durdur
5  print((stop-start)*1000)
```

✓ 0.0s

```
6.01823899989995
```

List ile Tuple arasındaki fark?

```
1  import sys
2  lis = [1,2,3,4]
3  tup = (1,2,3,4)
4  print("Liste kapladığı alan...: ", sys.getsizeof(lis))
5  print("Tuple  kapladığı alan..: ", sys.getsizeof(tup))
```

✓ 0.0s

```
Liste kapladığı alan...:  88
Tuple  kapladığı alan..:  72
```

Immutable özelliği

- Aşağıdaki programın çıktısı nedir?

```
1  city_lis = ["Tokyo", "Istanbul", "Moskova", "Ottowa", "Berlin"
2  city_tup = tuple(city_lis)
3  city_tup[1] = "Ankara"
4
```

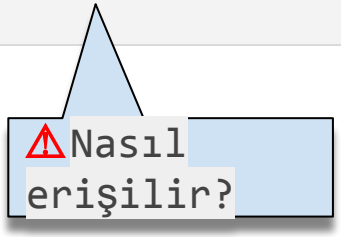
⊗ 0.3s

```
---------------------------------------------------------------------
TypeError                                 Traceback (most recent call
/home/bull/py_projects/colab_notebook_7_bulend.ipynb Cell 72 in ()
      1 city_lis = ["Tokyo", "Istanbul", "Moskova", "Ottowa", "Berlin
      2 city_tup = tuple(city_lis)
----> 3 city_tup[1] = "Ankara"
```

Tuple elemanlarına erişim

- Demet elemanlarına **indis** numarası ile erişilir. Buna göre mix_tuple'ın son elemanına nasıl erişebiliriz?

```
mix_tuple = (13, 'Yus', True, 3.14, None, (1, 2), (2, 'two'))
```





Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

Tuple elemanlarına erişim

- Demet elemanlarına **indis** numarası ile erişilir;

```
1  mix_tuple = (13, 'Yus', True, 3.14, None, (1, 2), (2, 'two'))
2  mix_tuple[-1]
```

✓ 0.0s

```
(2, 'two')
```

⚠Nasıl erişilir?

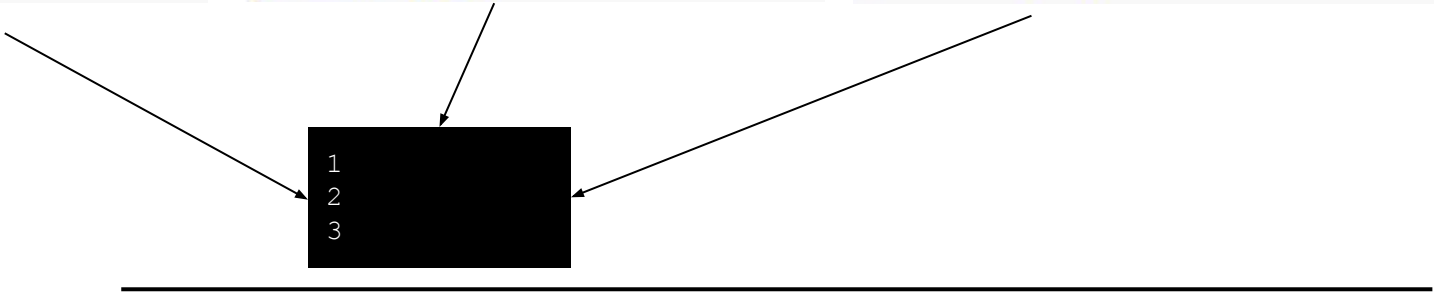
unpacking

- unpacking özelliği tüm iterable nesnelere uygulanabilir.

```
a, b, c = '123'
print(a)
print(b)
print(c)
```

```
a, b, c = (1, 2, 3)
print(a)
print(b)
print(c)
```

```
a, b, c = [1, 2, 3]
print(a)
print(b)
print(c)
```



unpacking

- unpacking özelliği tüm iterable nesnelere uygulanabilir.

```
2  a,b,*c = "alps"
3  print(a)
4  print(b)
5  print(c)
```

```
a
l
['p', 's']
```

```
1  a,*b,c = (1,2,3,4,5,6,7,8,9)
2  print(a)
3  print(b)
4  print(c)
5  print(type(b))
```

```
1
[2, 3, 4, 5, 6, 7, 8]
9
<class 'list'>
```

unpacking operatörleri: * , **

- Sözlüklerde **, diğer iterable nesnelerde * kullanılır.

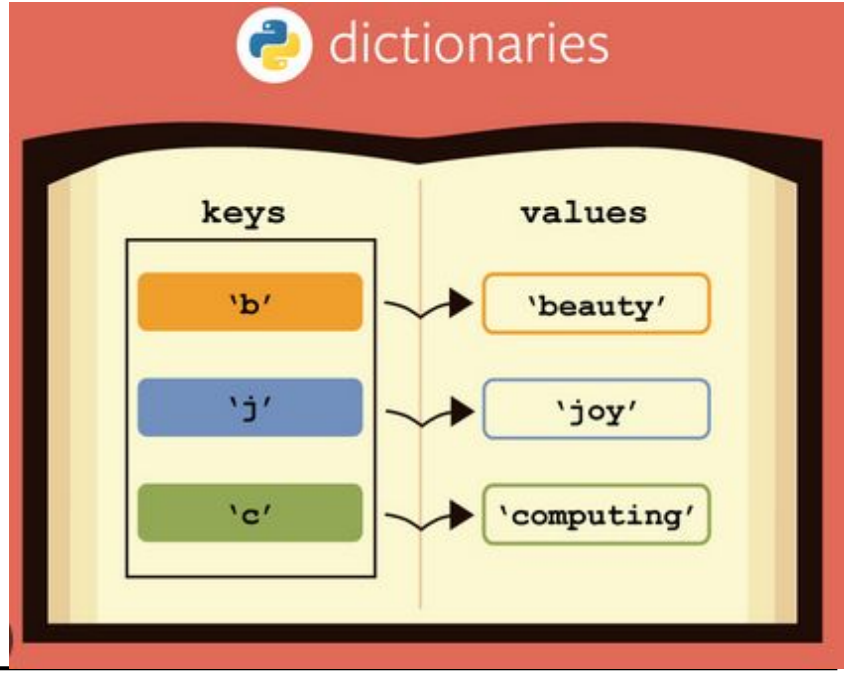
```
>>> print(*[1], *[2], 3)
1 2 3
>>> dict(**{'x': 1}, y=2, **{'z': 3})
{'x': 1, 'y': 2, 'z': 3}
```

```
>>> *range(4), 4
(0, 1, 2, 3, 4)
>>> [*range(4), 4]
[0, 1, 2, 3, 4]
>>> {*range(4), 4}
{0, 1, 2, 3, 4}
>>> {'x': 1, **{'y': 2}}
{'x': 1, 'y': 2}
```

dict() veri yapısı

- Gerçek dünyadaki sözlüklerin elektronik ortamdaki eşdeğeridir.

```
keys      :    values
{key1 : value1,
 key2 : value2}
```



dict() oluşturma

- Sözlükler anahtar(**key**)-değer(**value**) çiftlerinden oluşan veri yapılarıdır. Başka dillerde bunlara "lookup table", "**hash table**" ya da "**map**" ismi ile anılabilir.
- Boş bir sözlük 2 farklı şekilde tanımlanabilir;

```
bos = {}
bos = dict()
```

İki değişkenin çıktısı da aynıdır. Doğru mu?

```
musteri1 = {'meyve':'elma', 'icki':'süt'}
musteri2 = dict(meyve ='elma', icki='süt')
print(musteri1)
print(musteri2)
```



Students choose an option

Pear Deck Interactive Slide
Do not remove this bar

dict() elemanlarına erişim

- Sözlük elemanlarına erişim de indis kullanılmaz, key kullanılır.

```
aile= {
    'name1': 'Ali',
    'name2': 'Veli',
    'name3': 'Ayse'
    }
aile['name2']
```

key isimlendirmesi

- dict() ile sözlük oluştururken **key isimlerinde** kurallara uyulmalı!!!

```
aile = dict(name1 = 'Ali', name2 = 'Veli', name3= 'Ayse', name4 = 'Derya')
print(aile)
#{'name1': 'Ali', 'name2': 'Veli', 'name3': 'Ayse', 'name4': 'Derya'}
```

```
aile = dict(elif = 'Ali', name2 = 'Veli', name3= 'Ayse', name4 = 'Derya')
print(aile)
```

⊗ 0.0s

```
  Input In [4]
    aile = dict(elif = 'Ali', name2 = 'Veli', name3= 'Ayse', name4 = 'Derya')
                ^
SyntaxError: invalid syntax
```

Hepsinin çıktısı da aynı mıdır?

```
a = dict(one=1, two=2, three=3)
b = {'one': 1, 'two': 2, 'three': 3}
c = dict(zip(['one', 'two', 'three'], [1, 2, 3]))
d = dict([('two', 2), ('one', 1), ('three', 3)])
e = dict({'three': 3, 'one': 1, 'two': 2})
f = dict({'one': 1, 'three': 3}, two=2)
a == b == c == d == e == f
```



Students choose an option

Pear Deck Interactive Slide
Do not remove this bar

a sözlüğünü nasıl reverse ederiz?

```
a = dict(one=1, two=2, three=3)

print(a)
```



Students, write your response!

Pear Deck Interactive Slide
Do not remove this bar

- a sözlüğünü nasıl reverse ederiz?

```
1  a = dict(one=1, two=2, three=3)
2  print(a)
3  print(dict(reversed(a.items())))
```

✓ 0.0s

```
{'one': 1, 'two': 2, 'three': 3}
{'three': 3, 'two': 2, 'one': 1}
```

dict metotları

- `dir(dict) ile öğrenebiliriz.`

- Sözlük elemanlarına erişim için;
  - Her bir eleman çifti için **`.items()`**,
  - `Sadece key'ler için` **`.keys()`**,
  - `Sadece value'`lar için **`.values()`** metotları kullanılır.

https://docs.python.org/3/library/stdtypes.html#dict

dict metotları

`clear``()` :Sözlüğü boşaltır.

`copy``():` Sözlüğün bir kopyasını oluşturur.

`pop``(`*`key`*`[, `*`default`*`]):` `Sözlükte` *key* varsa onu siler ve değerini döndürür, yoksa *default* mesajı döndürür. Eğer *default* belirtilmez ise ve key'de sözlükte yoksa bir `KeyError` fırlatır.

`popitem``():` Sözlüğün en sonundaki `(key, value)` değer çiftini (elemanı) siler. Sözlük boşsa `KeyError` fırlatır.

`get``(`*`key`*`[, `*`default`*`]):` Aranan *key* sözlükte ise değerini yoksa *default değeri döndürür.* Default belirtilmezse, varsayılan değer None olur.

`setdefault``(`*`key`*`[, `*`default`*`]):` get ile aynı işleve sahiptir. Farkı aranan yoksa sözlüğe ekler.

`update``([`*`other`*`]):` Sözlüğü günceller. Örnek; `d.update(red=1, blue=2)`.

https://docs.python.org/3/library/stdtypes.html#dict

Sözlük Metotları



- **Task** 👇
  - Bir ingilizce-türkçe sözlük oluşturup, arama, güncelleme ve silme işlemlerini gerçekleştiriniz.

Bugünkü dersi anladık mı?



